Autonomous Perception & Planning for ADAS

A YOLO + BEV stack on nuScenes — architecture, algorithms, and business value

Technical Report

Contents

1. Abstract **2**

2. Problem statement and business value **2**

3. System overview **2**

4. Data pipeline (`data_preparation_v2.py`) **3**
- 4.1 Metadata resolution . . . 3
- 4.2 Ground-truth ego trajectory . . . 3
- 4.3 LiDAR → BEV rasterization . . . 3
- 4.4 Augmentation (training, `train_trajectory_v3.py`) . . . 3

5. Model 1 — YOLOv8 obstacle detector **3**

6. Model 2 — BEV multi-modal trajectory planner **4**
- 6.1 Architecture . . . 4
- 6.2 Multi-modal formulation . . . 4
- 6.3 Training objective . . . 4
- 6.4 Evaluation metrics . . . 5

7. Recovering a lost model from its weights **5**

8. Collision-risk fusion (`collision_risk.py`) **5**
- 8.1 Obstacle map . . . 5
- 8.2 Corridor scan . . . 5
- 8.3 Speed, TTC, and decision . . . 5
- 8.4 Validation . . . 6

9. Camera–LiDAR synchronisation (`scene_dashboard.py`) **6**

10. VLM reasoning layer (`vlm_reasoner.py`, `vlm_analyze.py`) **6**
- 10.1 Inputs . . . 6
- 10.2 Task and output . . . 6
- 10.3 Backend . . . 7
- 10.4 Why it matters . . . 7
- 10.5 Narrated-scene demo (`scene_narrator.py`) . . . 7

11. Inference and delivery **7**

12. Engineering practices **7**

13. Limitations and future work **7**

14. Technology stack **8**

15. Key figures **8**

1. Abstract

This report documents an end-to-end autonomous-driving perception and motion-planning system built on the nuScenes dataset and trained on AWS SageMaker. It combines a **YOLOv8 camera object detector**, a **custom bird's-eye-view (BEV) multi-modal trajectory planner** operating on LiDAR, and a **collision-risk fusion layer** that converts those two model outputs into an actionable CLEAR / CAUTION / BRAKE decision with distance-to-obstacle and time-to-obstacle (TTC). The system is delivered as an interactive Streamlit application, a sequence-level risk-monitor video generator, and a per-scene PDF safety report. This document covers the data pipeline, both model architectures, the fusion algorithm, the camera–LiDAR synchronisation, the engineering practices used (including reconstructing a lost model definition from its weights), the measured results, and the business rationale.

2. Problem statement and business value

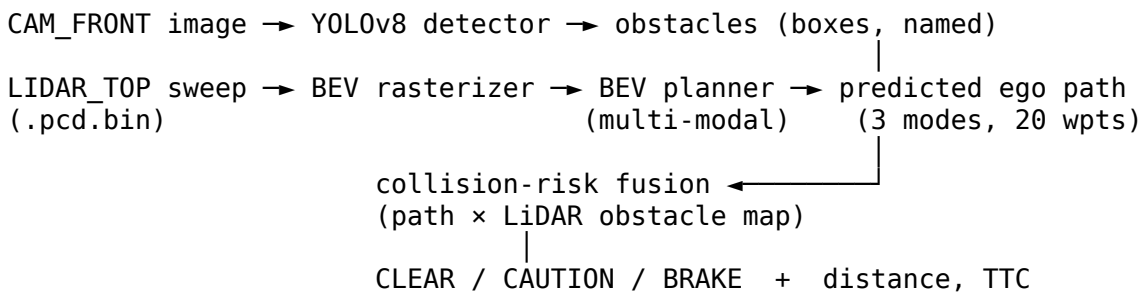
Forward-collision warning (FCW) and automatic emergency braking (AEB) are increasingly mandated and incentivised by safety regulators worldwide. The underlying technical capability is a closed loop: **perceive the surrounding scene, predict the ego vehicle's near-future motion, and flag any conflict between the two.**

The same fused module is the input to three distinct commercial products, which is what makes it valuable to build well once:

Market	Consumes	Concrete output
ADAS / OEM safety systems	a real-time warn/brake trigger	per-frame CLEAR/CAUTION/BRAKE + TTC
Fleet management & usage-based insurance	a per-drive risk score	risk timeline + ranked near-miss events
AV research & data operations	scene triage and auto-labelling	multi-modal trajectories + scene risk profile

The decisive quantities — **distance to the nearest in-path obstacle** and **time to reach it** — are common to all three. This project therefore optimises for a correct, reproducible decision pipeline rather than for a single benchmark metric.

3. System overview



Two sensors, two networks, one decision surface. The camera detector establishes *what* is present; the BEV planner establishes *where the ego is going*; the fusion layer establishes *whether that constitutes a hazard*.

4. Data pipeline (`data_preparation_v2.py`)

Development used the full ~200 GB nuScenes release on an AWS SageMaker notebook instance. nuScenes stores its annotations as a relational set of JSON tables (`sample`, `sample_data`, `ego_pose`, `calibrated_sensor`, `sample_annotation`, `instance`, `category`). The pipeline performs four stages.

4.1 Metadata resolution

Each keyframe `sample` is linked to its `LIDAR_TOP` `sample_data` record (filtered on `channel == 'LIDAR_TOP'` and `is_key_frame`), its `ego_pose`, and its 3-D box annotations with resolved category names.

4.2 Ground-truth ego trajectory

The supervision signal is the ego vehicle's own future path. Future keyframe ego positions are read, then transformed from the global frame into the **current ego frame** so the trajectory is expressed relative to the car "now". For a future global position `p_world`, current ego translation `t` and current ego yaw rotation R:

$$p_{ego} = R^{-1} \cdot (p_{world} - t)$$

The resulting variable-length path is interpolated to a fixed **20 waypoints** in `(x, y)` (x = forward, y = left), giving a consistent regression target.

4.3 LiDAR → BEV rasterization

Each LiDAR point cloud (nuScenes stores 5 floats/point: x, y, z, intensity, ring) is filtered to a region of interest and projected onto a top-down grid:

- ROI: $x, y \in [-50, 50]$ m, $z \in [-3, 3]$ m; resolution $r = 0.2$ m/cell → a **500 × 500** grid.
- Grid index: $gx = \lfloor (x + 50)/0.2 \rfloor$, $gy = \lfloor (y + 50)/0.2 \rfloor$.
- **Channel 0 — height:** per-cell max of normalised z, $(z + 3)/6$.
- **Channel 1 — intensity:** per-cell max of $\min(\text{intensity}/255, 1)$.
- **Channel 2 — density:** point count per cell, clipped to $\text{count}/10 \in [0, 1]$.

The output tensor is `(500, 500, 3)`, permuted to `(3, 500, 500)` for PyTorch.

4.4 Augmentation (training, `train_trajectory_v3.py`)

Because most driving is straight, a naive model collapses to "go straight". To force it to learn turns, training applies, jointly to the BEV raster and the trajectory: **random rotation** (±25°, applied as a 2-D rotation matrix to the waypoints and an image rotation to the BEV) and **random horizontal flip** (mirror y and flip the BEV), turning left manoeuvres into right ones and vice versa.

5. Model 1 — YOLOv8 obstacle detector

A YOLOv8-nano detector fine-tuned with Ultralytics. Training configuration (from `runs/train/nuscenes_yolo/ar`
base `yolov8n.pt`, image size 640, batch 16, 20 epochs, auto optimizer, **device = CPU**, single class (`nc = 1, names = ['object']`), 20,256 training images.

Single-class rationale. Collapsing all road objects to one `object` class keeps the downstream warning logic trivial — any sufficiently-confident detection is a hazard — and halves the labelling/learning burden on a CPU budget.

Named detection without retraining. Where object *type* matters, the system swaps in a stock **COCO-pretrained YOLOv8** restricted to road classes (ids `0,1,2,3,5,7` = person, bicycle, car, motorcycle, bus, truck). nuScenes frames are ordinary street scenes the COCO model handles directly, so named detection is available at zero training cost.

Metrics (epoch 20). Precision 0.64, Recall 0.30, mAP@0.50 0.35, mAP@0.50–0.95 0.21. mAP@0.50 rose monotonically from 0.04 (epoch 1) to 0.35 (epoch 20) and had not plateaued — the detector is compute-limited, not converged.

6. Model 2 — BEV multi-modal trajectory planner

6.1 Architecture

Input `(3, 500, 500)` BEV → feature encoder → 256-d embedding → four heads. ~12M parameters.

Block	Layers	Output
Encoder stem	Conv 7×7 s2 (3→64), BN, ReLU, MaxPool 3×3 s2	64 ch
Encoder layer1	2 × (Conv-BN-ReLU-Conv-BN), downsample	128 ch
Encoder layer2	2 × double-conv, downsample	256 ch
Encoder layer3	2 × double-conv, downsample	512 ch
Embedding	GlobalAvgPool → Linear(512→256) → ReLU	256-d
Trajectory heads	3 × [Linear(256→256), ReLU, Dropout, Linear(256→40)]	3 × (20×2)
Mode head	Linear(256→128), ReLU, Linear(128→3) → softmax	3 probs
Confidence head	Linear(256→128), ReLU, Linear(128→20)	20 (sigmoid)
Segmentation head	4 × ConvTranspose (512→256→128→64→32), Conv 1×1 → 8	8-class BEV map

6.2 Multi-modal formulation

Predicting one path hides risk: at a junction several futures are plausible. The network therefore emits `M = 3` candidate trajectories `{τ_m}`, each 20 waypoints, with a categorical mode distribution `π = softmax(mode_logits)`. The selected path is `τ* = τ_argmax(π)`. A per-waypoint confidence and an auxiliary 8-class BEV semantic segmentation (background, car, truck, bus, bicycle, motorcycle, pedestrian, movable-object) regularise the encoder.

6.3 Training objective

The v3 recipe combines a **Huber (smooth-L1) regression loss** on the best-matching mode (robust to outlier waypoints), a **mode-classification loss**, and a **diversity term** that penalises modes collapsing onto one another — directly countering the straight-line failure mode — plus the auxiliary segmentation loss, with learning-rate warmup.

6.4 Evaluation metrics

Standard motion-forecasting metrics, over `T = 20` waypoints with prediction `p_t` and ground truth `g_t`:

```
ADE = (1/T) Σ_t ‖ p_t − g_t ‖₂        (average displacement error)
FDE = ‖ p_T − g_T ‖₂                   (final displacement error)
```

Checkpoint metadata (epoch 5): **val ADE 3.29 m, val FDE 7.23 m, val loss 3.83**. This is an early checkpoint with a sensible "continue forward" prior but limited turn commitment.

7. Recovering a lost model from its weights

The original `trajectory_model.py` class definition was lost with the SageMaker instance; only the 146 MB checkpoint survived. The architecture was reconstructed by:

1. Reading the checkpoint's `state_dict` and extracting every parameter's **name and tensor shape** (a custom `pickle` unpickler stubbing out the tensor-rebuild ops, so no GPU/torch build was needed).
2. Inferring each module from its key path and shapes (e.g. `bev_encoder.layer1.0.0.weight` of shape `(128, 64, 3, 3)` → first conv of a double-conv block, 64→128).
3. Rebuilding the `nn.Module` so it reproduces **all 130 parameter tensors** with identical names and shapes — verified by a set-difference check (0 missing, 0 unexpected, 0 shape mismatches), guaranteeing `load_state_dict(strict=True)` succeeds.

The reconstructed class also matched the original output visualisations independently (3 modes, 20-waypoint confidence, the 8 segmentation classes), confirming correctness.

8. Collision-risk fusion (`collision_risk.py`)

The fusion layer is pure NumPy (unit-testable, no torch) and converts a predicted path plus the BEV raster into a decision.

8.1 Obstacle map

A BEV cell is an obstacle iff it is **dense and elevated**:

```
obstacle(c) = (density(c) > 0.2) AND (height(c) > 0.55)
```

The height test removes the flat road surface, which a density-only map would keep.

8.2 Corridor scan

The predicted path is resampled by arc length and walked outward from the ego. At each point a half-vehicle-width corridor (±1.0 m → ±5 cells) is tested against the obstacle map, with an **ego deadzone** (`< 3 m`) ignoring the vehicle's own returns. The first hit gives the nearest in-path obstacle distance `d`.

```
for waypoint (x, y) along path, increasing range:
    if range < 3 m: continue                      # ego deadzone
    if any obstacle cell within ±1 m corridor:    # vehicle-width
        d = range(x, y); break
```

8.3 Speed, TTC, and decision

Ego speed is estimated from the predicted path length over its time horizon (`horizon = 20 × 0.5 s`, since nuScenes keyframes are 0.5 s apart):

```
v = path_length / horizon
TTC = d / v
```

Decision thresholds: `d < 8 m → BRAKE`, `8 ≤ d < 20 m → CAUTION`, else `CLEAR`.

8.4 Validation

On a 120-frame sample scene the logic produces a realistic spread of decisions with closest approaches of **3.5–33 m** (not all-clear, not all-collision), confirming the geometry and thresholds behave sensibly on real LiDAR.

9. Camera–LiDAR synchronisation (`scene_dashboard.py`)

The dashboard pairs the front camera with the BEV view. Because the bundled package ships **without nuScenes calibration matrices** (camera intrinsics K and the LiDAR→camera extrinsic transform), a true point-on-image projection is not possible; the field-standard multi-panel layout is used instead (as the nuScenes devkit itself renders).

Synchronisation is exact in **time**, however, using filename timestamps:

- **Timestamp parse** — the trailing microsecond integer in each nuScenes filename.
- **Scene grouping** — a time-sorted frame list is split wherever the inter-frame gap exceeds 2 s, yielding the dataset's discrete scenes (67 in the bundled LiDAR set).
- **Front-camera filter** — keep only the exact `CAM_FRONT` token (excluding `CAM_FRONT_LEFT/_RIGHT` and the rear cameras), so the dashboard never mixes viewing directions.
- **Nearest-timestamp match** — each selected LiDAR frame is paired to the front-camera frame with the closest timestamp (within ±0.5 s), giving frame-accurate sync.

A `project_lidar_to_image()` function is included as the explicit upgrade path: supply K and the 4×4 transform and the dashboard switches to a true depth-coloured LiDAR overlay (`uv = K · [R|t] · p`, keeping points with positive camera-frame depth).

10. VLM reasoning layer (`vlm_reasoner.py`, `vlm_analyze.py`)

The geometric pipeline is closed-vocabulary and non-explanatory. A local Vision-Language Model (VLM) adds an open-vocabulary semantic layer that reasons over the entire sensor suite.

10.1 Inputs

The VLM receives one composite image — a 2×3 montage of the six nuScenes cameras (`CAM_FRONT_LEFT/FRONT/FRONT_RIGHT`, `CAM_BACK_LEFT/BACK/BACK_RIGHT`) plus the rendered LiDAR BEV tile — together with a text summary of the geometric stack's output (risk level, nearest in-path obstacle, TTC, ego speed, per-camera detections). Cameras are paired to the LiDAR frame by nearest filename timestamp (±0.5 s); the LiDAR is supplied as the BEV render because VLMs cannot ingest raw point clouds. Reasoning over the full surround — not just the front camera the driver already sees — is the point: the model synthesises views no single human perspective provides.

10.2 Task and output

A constrained prompt casts the model as the high-level ADAS reasoning module and requests strict JSON (Ollama `format=json`): a scene caption, an open-vocabulary hazard list, agreement with the CLEAR/CAUTION/BRAKE decision, a rationale, and a recommended action. The hazard vocabulary deliberately targets what a bounding-box detector cannot label — construction zones, emergency vehicles, traffic-light state, debris, animals, vulnerable road users, and road/weather conditions.

10.3 Backend

`OllamaVLM` posts to a local Ollama server (`/api/generate`, base64 images). The default model is `qwen2.5vl:7b` (Qwen2.5-VL, multi-image). Text-only models such as `qwen3:8b` are auto-detected and receive only the text summary (no images). A `MockVLM` returns schema-valid output so the full pipeline — montage assembly, PNG encoding, prompt construction, JSON parsing, card rendering — is testable with no GPU and no server.

10.4 Why it matters

This converts a purely geometric system into a **neuro-symbolic** one: geometry supplies precise, calibrated measurement (distance, TTC); the VLM supplies meaning (what the obstacle is, what the scene context implies) and a human-readable justification — an independent corroboration channel and the foundation for explainable, auditable ADAS decisions.

10.5 Narrated-scene demo (`scene_narrator.py`)

The LiDAR stream is split into scenes by timestamp gaps; 4–5 are auto-selected for risk variety (clear / caution / brake). For each scene the system builds, per frame, the 6-camera surround montage plus the LiDAR BEV, then queries the VLM for a short live "co-pilot" comment per temporal segment and a holistic scene summary over several keyframes (a single multi-image call). The result is a subtitled clip (GIF + MP4) with a per-scene transcript (scene type, summary, hazards, segment commentary) and an index page. This is the portfolio centrepiece: a local model narrating real driving scenes from full surround + LiDAR context.

11. Inference and delivery

- **`app.py`** — Streamlit app with three tabs: detection (fine-tuned/named), single-frame trajectory + risk, and the multi-frame scene dashboard (scene/range selection, CAM_FRONT folder, timestamp pairing) exporting an animated GIF.
- **`run_inference.py`** — batch inference (`yolo` / `named` / `bev` / `all`).
- **`risk_monitor.py`** — sequence → annotated MP4/GIF risk monitor.
- **`safety_report.py`** — sequence → per-scene PDF (safety score, risk composition, timeline, ranked near-miss events).

All inference runs on a laptop CPU against bundled sample frames; the heavy training ran on SageMaker. Trained weights are versioned with Git LFS.

12. Engineering practices

- **Reproducibility** — relative paths, pinned `requirements.txt`, one-command entry points.
- **Honesty** — every reported number is read from the training logs / checkpoint metadata; the detector's under-training and the planner's early-checkpoint straight-line bias are documented rather than hidden.
- **Testability** — the risk fusion and synchronisation logic are pure NumPy and validated directly on real frames.
- **Recoverability** — a lost model architecture was rebuilt from weights and formally verified.

13. Limitations and future work

1. **Planner maturity** — epoch-5 checkpoint; resuming training with the v3 augmentation / diversity recipe is the highest-leverage next step (lower FDE, real turn commitment).

2. **Detector** — single-class, CPU-trained, still improving at epoch 20; multi-class GPU training would raise mAP and enable typed hazards end-to-end.
3. **Cross-sensor fusion** — risk currently uses the LiDAR obstacle map only; with calibration, camera detections can be projected into the BEV frame for one fused map.
4. **Deployment** — latency profiling and ONNX/TensorRT export for an embedded target.

14. Technology stack

Layer	Tools
Modelling	PyTorch, Ultralytics YOLOv8
VLM reasoning	Ollama · Qwen2.5-VL (local)
Data	nuScenes, NumPy, custom BEV rasterizer
Training	AWS SageMaker (notebook instances)
Fusion / analytics	NumPy, SciPy
Visualisation / delivery	Matplotlib, OpenCV, Pillow, Streamlit
Reporting	Matplotlib PdfPages, pandoc

15. Key figures

Item	Value
nuScenes raw dataset	~200 GB
YOLO training images	20,256
Bundled LiDAR frames / scenes	3,376 / 67
BEV raster	500 × 500 × 3
Planner parameters / tensors	~12M / 130
Trajectory modes / horizon	3 / 20 waypoints (~ a few seconds)
YOLO mAP@0.50 / @0.50–0.95	0.35 / 0.21
Planner val ADE / FDE	3.29 m / 7.23 m